CS7646

Xing Liu
xliu397@gatech.edu

***Abstract*—** This report investigates the behavior of Classification and Regression Tree (CART) learners. Three experiments evaluate how the leaf_size hyperparameter affects overfitting in Decision Tree learners, whether bagging mitigates overfitting, and how Decision Trees compare to Random Trees in terms of model complexity and prediction error.

1 INTRODUCTION

Decision trees are a fundamental lass of supervised learning algorithms that recurseively partition feature space to produce predictions. A key hyperparameter in decision tree construction is leaf_size, which controls the minimum number of samples required at a node before it becomes a leaf. Small leaf_size values produce deep, complex trees that fit training data closely but may generalize poorly on unseen data due to overfitting. Conversely, large leaf_size values produce shallow trees that underfit the data.

This report uses the Istanbul Stock Exchange dataset, which contains daily returns for multiple indexes. The prediction ask is to estimate the index return based on the returns of other indexes. Three experiments are conducted:

1. Evaluating how leaf_size affects overfitting in a Decision Tree Learner using RMSE.
2. Investigating whether bagging can reduce or eliminate overfitting in Decision Tree Learner using RMSE.
3. Comparing Decision Tree and Random Tree Learner using max error and model build time.

The following hypothesis guide the investigation

1. Overfitting occurs at small leaf-size values, where in-sample error is low but out-of-sample error is high
2. Bagging reduces overfitting by averaging predictions across multiple trees trained on different samples, but may not fully eliminate it.

3. Decision Tree Learners produce lower max errors but take longer to build than Random Tree Learners due to having to find correlations rather than picking a random split value.

1.1 Methods

Dataset: The Istanbul Stock Exchange dataset (Istanbul.csv) contains 536 observations with 8 numerical feature columns representing returns of various worldwide indexes. The target variable (Y) is the MSCI Emerging Markets index return (the last column). The date column (first column) and header row are excluded during data loading.

Data Split: The dataset is randomly shuffled, then split into 60% training data (321 observations) and 40% test data (215 observations).

Experiment 1: Overfitting in DTLearner: A DTLearner is trained for each leaf_size value from 1 to 49 (inclusive). For each configuration, both in-sample and out-of-sample RMSE are computed. RMSE is defined as sqrt(mean((y_actual - y_predicted)^2)). Overfitting is identified where in-sample RMSE is substantially lower than out-of-sample RMSE.

Experiment 2: Effect of Bagging on Overfitting: A BagLearner wrapping DTLearner with 10 bags is trained for each leaf_size from 1 to 49. Each bag trains on a bootstrap sample (sampling with replacement) of the same size as the training set. The final prediction is the mean of all bag predictions. In-sample and out-of-sample RMSE are computed for each leaf_size.

Experiment 3: DTLearner vs RTLearner: Both learners are trained for each leaf_size from 1 to 49. Two metrics are compared:

(a) Time to Train: time in seconds to build the tree via add_evidence().
(b) Maximum Error (ME): the largest absolute difference between predicted and actual value on the out-of-sample test set, i.e., max(|y_actual - y_predicted|). This captures worst-case prediction behavior.

2 DISCUSSION

2.1 Experiment 1

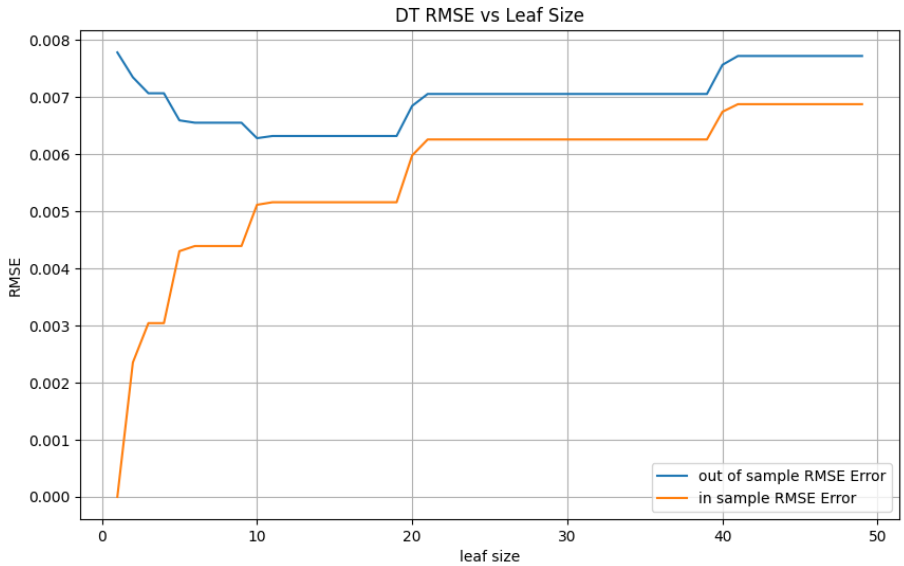

Figure 1 — plots in-sample and out-of-sample RMSE for DTLearner across leaf_size values from 1 to 49. Overfitting clearly occurs with respect to leaf_size:

Overfitting occurs when a model captures noise in the training data rather than the underlying signal, resulting in strong in-sample performance but poor out-of-sample generalization. It matters because an overfit model gives misleadingly optimistic training metrics while failing on new data. Overfitting is typically caused by excessive model complexity—in the case of decision trees, trees that are too deep with too many leaves. Common mitigation strategies include limiting tree depth (increasing leaf_size), pruning, cross-validation for hyperparameter selection, and ensemble methods such as bagging.

As leaf_size increases from 1, the in-sample RMSE rises while the out-of-sample RMSE decreases. The out-of-sample RMSE reaches its approximate minimum around leaf_size = 8 to 12, at roughly 0.0062. This range represents the optimal hyperparameter setting—the best balance between model complexity and generalization.

The starting point of overfitting is at approximately leaf_size = 10, and the direction of overfitting is toward decreasing leaf_size values. That is, overfitting

worsens as leaf_size decreases from ~10 down to 1. In this region, the in-sample RMSE plummets toward zero while the out-of-sample RMSE climbs, widening the gap.

Beyond leaf_size ~18, both in-sample and out-of-sample RMSE converge and increase together, indicating the onset of underfitting. At leaf_size = 40, both errors are approximately 0.007-0.008.

2.2 Experiment 2

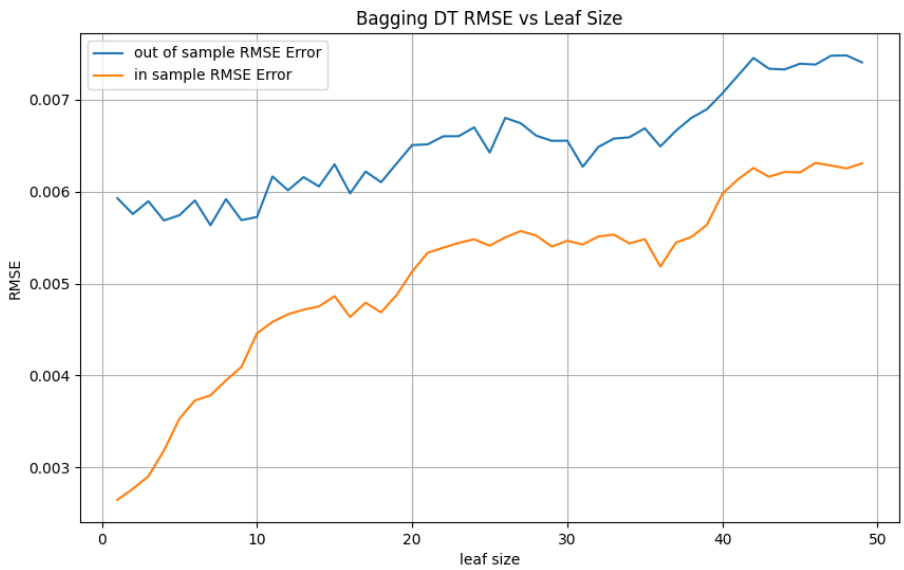

Figure 2 — in-sample and out-of-sample RMSE for a BagLearner containing 10 DTLearners, across leaf_size from 1 to 49.

Bagging reduced the overfitting. The out-of-sample RMSE at leaf_size = 1 also improved, dropping from ~0.0077 to ~0.0058 (~25% reduction).

This reduction occurs because each bag trains on a different bootstrap sample of the data. No single tree sees the entire training set, so individual trees cannot perfectly memorize it. Averaging predictions across 10 differently-trained trees smooths out the noise each tree captures, reducing variance and improving generalization.

Even with 10 bags, a gap between in-sample and out-of-sample RMSE persists. At leaf_size = 1, the gap is still approximately 0.003. The in-sample RMSE (~0.0025) remains clearly below the out-of-sample RMSE (~0.006). Some

overfitting still occurs at small leaf_size values because each individual base learner remains overfit to its bootstrap sample, and averaging overfit models reduces but does not fully eliminate the effect.

The optimal leaf_size with bagging is approximately 5 to 10, where the out-of-sample RMSE is minimized around 0.0058-0.006. Overfitting begins from this optimal range and worsens in the direction of decreasing leaf_size, similar to Experiment 1 but with a much-reduced severity.

2.3 Experiment 3

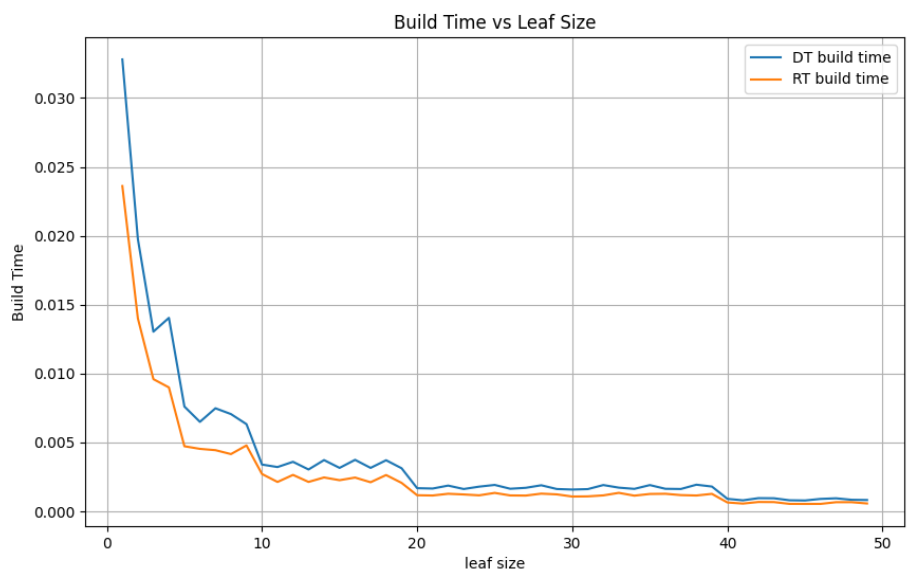

Figure 3 — plots the time (in seconds) to train each learner as a function of leaf_size.

DTLearner is consistently slower than RTLearner across all leaf_size values. At leaf_size = 1, DTLearner takes approximately 0.033 seconds to train, while RTLearner takes approximately 0.024 seconds—DTLearner is roughly 38% slower.

This difference is caused by the feature selection mechanism. At every internal node during tree construction, DTLearner must compute the absolute correlation between each candidate feature and the target variable Y, then select the feature with the highest correlation. This involves O(n * d) computation at each node

(where n is the number of samples and d is the number of features). RTLearner, by contrast, simply generates a single random integer to select a feature, which is an O(1) operation. The cumulative savings across hundreds of nodes result in meaningfully faster training.

Both learners show decreasing training time as leaf_size increases, because larger leaf_size produces shallower trees with fewer nodes—and therefore fewer split computations. The gap narrows at large leaf_size values (>20) where both trees are small and the correlation overhead becomes negligible.

RTLearner is better in this regard: it trains faster, which is advantageous when training time is a constraint, such as in ensemble settings where many trees are constructed (e.g., random forests with hundreds of bags).

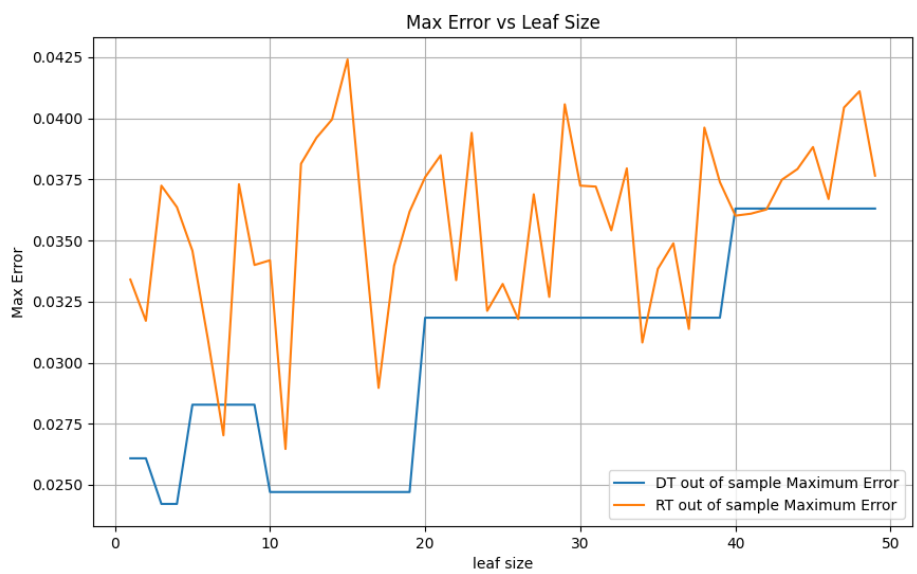

Figure 4 — plots the maximum out-of-sample prediction error—the single worst-case absolute error across all test observations—for each learner.

DTLearner produces substantially lower and more stable maximum errors than RTLearner. DTLearner's out-of-sample max error ranges from approximately 0.025 to 0.036 across all leaf_size values, with a smooth, predictable curve. RTLearner's max error ranges from approximately 0.027 to 0.0425, with extreme volatility—sharp spikes and dips across adjacent leaf_size values.

This difference arises because DTLearner's correlation-based feature selection identifies the most informative feature at each split, producing partitions that separate data points in a way that is well-aligned with the target variable. This leads to more reliable leaf predictions and fewer extreme mispredictions. RTLearner's random feature selection sometimes picks uninformative or weakly relevant features, creating suboptimal splits that can produce leaves with poorly representative averages—leading to larger errors for specific test points.

The volatility in RTLearner's max error curve is also notable. Because the feature choice is random, small changes in leaf_size can result in very different tree structures, leading to inconsistent worst-case behavior. DTLearner's deterministic approach produces consistent tree structures for a given leaf_size, yielding a smoother curve.

DTLearner is better in this regard: it provides more predictable and lower worst-case errors. In applications where reliability matters—such as financial prediction where a single extreme misprediction could trigger a poor trading decision—DTLearner's stability is a meaningful advantage.

Overall Comparison:

1. RTLearner is better in terms of training speed (approximately 30-40% faster at small leaf sizes) due to avoiding correlation computation.
2. DTLearner is better in terms of worst-case prediction reliability (lower and more stable maximum error) because its informed splits produce more reliable leaf predictions.
3. Neither learner is universally superior. DTLearner offers more reliable predictions at the cost of slower training. RTLearner is faster to train but produces less predictable worst-case errors. In practice, the choice depends on the application: when many trees must be trained quickly (as in large ensembles), RTLearner's speed advantage is valuable. When prediction reliability and robustness matter more than training speed, DTLearner is the stronger choice.

3 SUMMARY

Three experiments investigated CART learner behavior on the Istanbul Stock Exchange dataset.

First, overfitting in DTLearner was clearly demonstrated as a function of leaf_size. The optimal leaf_size was approximately 8 to 12, and overfitting intensified as leaf_size decreased below this range toward 1. At the smallest leaf_size, the tree memorized training data (near-zero in-sample RMSE) while performing poorly on unseen data (out-of-sample RMSE ~0.0078).

Second, bagging with 10 bags reduced overfitting substantially—cutting the in-sample/out-of-sample gap from ~0.008 to ~0.003 at leaf_size = 1. However, bagging did not fully eliminate overfitting; a residual gap remained at small leaf_size values.

Third, DTLearner and RTLearner exhibited a clear tradeoff. RTLearner trained approximately 30-40% faster due to random (rather than correlation-based) feature selection, but DTLearner produced lower and more stable worst-case prediction errors. Neither learner was universally superior; their relative value depends on whether training speed or prediction reliability is prioritized.